

Build a Text Generative and Image Generative Model

*Report Submitted to K RAMAKRISHNAN COLLEGE OF
ENGINEERING, TRICHY*

Submitted by

SATHYABALAN S

8115U23AM045

JULY 2025



**K.RAMAKRISHNAN
COLLEGE OF ENGINEERING**
An Autonomous Institution

Permanently Affiliated to Anna University Chennai, Approved by AICTE New Delhi,
ISO 9001:2015, 14001:2015 certified institution, Accredited by NBA and with A grade by NAAC

Samayapuram, Tiruchirappalli – 621 112, Tamilnadu, India.



K.RAMAKRISHNAN COLLEGE OF ENGINEERING

An Autonomous Institution

Permanently Affiliated to Anna University Chennai, Approved by AICTE New Delhi,
ISO 9001:2015, 14001:2015 certified institution, Accredited by NBA and with A grade by NAAC

Samayapuram, Tiruchirappalli – 621 112, Tamilnadu, India.

Bonafide Certificate

This is to certify that the report titled "*Build a Text Generative and Image Generative Model*" project, completed during my internship period, is a bonafide record of the work done by **SATHYABALAN S (8115U23AM045)** during the academic year 2024 - 2025, from K Ramakrishnan College of Engineering, Trichy, under my supervision.

Signature of Mentor :

Name with Affiliation : **Premkumar Nagendran**

Director, Vanigam Software Services Private Limited

Date : **19.07.2025**

Project *Viva voice* held on _____

Examiner 1

Examiner 2



K.RAMAKRISHNAN COLLEGE OF ENGINEERING

An Autonomous Institution

Permanently Affiliated to Anna University Chennai, Approved by AICTE New Delhi,
ISO 9001:2015, 14001:2015 certified institution, Accredited by NBA and with A grade by NAAC

Samayapuram, Tiruchirappalli – 621 112, Tamilnadu, India.

Declaration

I declare that the report titled “**Build a Text generative and Image generative model**” submitted by me is an original work done by me under the guidance of **Premkumar Nagendran** during the fifth semester of the academic year 2024 -2025, in the **K Ramakrishnan College of Engineering, Trichy**. The work is original and wherever I have used materials from other sources, I have given due credit and cited them in the text of the report.

Signature of the candidate(s) :

Name of the candidate(s) : SATHYABALAN S

Date :19.07.2025

Acknowledgements

I wish to address my profound gratitude to **Dr. K. RAMAKRISHNAN**, Chairman, K. Ramakrishnan College of Engineering (Autonomous), who encouraged and gave me all help throughout the course.

I extend my hearty gratitude and thanks to my honorable and grateful Executive Director **Dr.S.KUPPUSAMY, B.Sc., MBA., Ph.D.,** K. Ramakrishnan College of Engineering(Autonomous).

I am glad to thank my Principal **Dr.D.SRINIVASAN, M.E.,Ph.D., FIE., MIW., MISTE., MISAE., C.Engg,** for giving me permission to carry out this project.

I wish to convey my sincere thanks to **Dr.B.KIRAN BALA, M.E., M.B.A., Ph.D.,** Head of the Department, Artificial Intelligence and Data Science for giving me constant encouragement and advice throughout the course.

I am grateful to my project mentor **Premkumar Nagendran**, Director of Vanigam Software Services Private Limited for his valuable suggestions, guidance, constant supervision and supporting me in all stages for the successful completion of this project.

Finally, I sincerely acknowledge in no less terms all my staff members, my parents and friends for their co-operation and help at various stages of this project work.

Table of Contents

Title.....	1
Bona-fide Certificate.....	2
Declaration.....	3
Acknowledgements.....	4
List of Figures.....	6
Abstract.....	7
1. Introduction.....	8
1.1 Problem Statement.....	8
1.2 Objectives.....	9
2. Basics.....	9
2.1 Understanding the Project.....	9
3. Implementation.....	10
3.1 Model selection	10
3.2 Python code.....	11
4. Fine Tuning.....	14
4.1 Implementation of fine tuning	15
4.2 Code & Result	16
5. Image Generative IIm	20
5.1 Code	20
5.2 Result	21
6. Conclusion.....	22
6.1 Conclusion.....	22
6.2 What I learned?	22
References.....	23

List of Figures

S. No	Figure Name	Page No.
1	Text generative model	15
2	Fine tuned llm result page	20
3	Image generative model	22

ABSTRACT

During this internship, I completed a course titled Building Intelligent Chatbots using AI, where I acquired skills in developing AI-powered chatbots and leveraging Streamlit to create interactive web pages using Python. Additionally, I revisited an Excel course to generate a manual dataset for fine-tuning Large Language Models (LLMs). This experience encompassed learning the fundamentals of AI application development, including the use of Python-based frameworks for building intelligent systems. Streamlit enabled the creation of user-friendly web interfaces, while the Excel dataset supported customized model training. The internship provided hands-on experience in integrating AI technologies and web development tools, fostering a practical understanding of building and deploying intelligent applications for real-world use cases.

Chapter 1

1. INTRODUCTION

This project focuses on Large Language Models (LLMs), which are powerful AI tools designed to understand and generate human-like text. Our aim is to develop a system that not only utilizes these advanced models but also makes them easily accessible and customizable. This involves carefully selecting the right LLM, then fine-tuning it with specific data to make it unique and more accurate for particular tasks, and finally, making it available through a user-friendly web interface. By doing this, we can help companies get quick answers and assistance, allowing them to manage their information more efficiently. Fine-tuning, though challenging with lots of data, is key to making an LLM truly belong to a company and serve its specific needs.

1.1 PROBLEM STATEMENT

A significant challenge in developing an effective Large Language Model (LLM) lies in improving the accuracy of the model's responses. This hinges critically on choosing the right base model for the intended application. Furthermore, fine-tuning the chosen model using a manually curated dataset is a really complex process. This complexity arises from the vastness of data, the intricacies of labeling, and the computational demands involved in tailoring the LLM to deliver precise and relevant outputs for specific needs.

1.2 OBJECTIVES

This project aims to strategically design, develop, and deploy a robust and highly accurate Large Language Model (LLM) system, engineered to deliver transformative benefits across various applications. The core methodology involves the meticulous selection of an optimal foundational LLM, followed by its precise fine-tuning utilizing unique, manually curated, and domain-specific datasets. This rigorous process is geared towards overcoming the inherent complexities of tailoring general-purpose models, ensuring the generation of contextually relevant and reliable responses.

Ultimately, this initiative seeks to establish a cutting-edge, customizable AI solution that significantly enhances organizational efficiency, streamlines intricate data management processes, and provides intelligent, real-time assistance, thereby empowering users with improved insights and fostering more informed decision-making across diverse operational landscapes.

Chapter 2

2. BASICS

A Large Language Model (LLM) is like a super-smart computer program that has read and learned from an enormous amount of text – think of it as having read nearly every book, article, and website on the internet! Because of all this reading, an LLM can understand human language and also create new text that sounds very natural. When you ask an LLM a question, tell it to write a story, or even summarize a long document, it uses all that knowledge to give you a helpful answer. It's essentially a very advanced text predictor, guessing the most likely next word based on everything it has learned, which makes it incredibly versatile for tasks like writing, translating, and answering questions.

2.1 UNDERSTANDING THE PROJECT

This project is all about making Large Language Models (LLMs) truly useful and easy for companies to use. We're building a system that takes these powerful AI tools – which are great at understanding and generating human-like text – and customizes them for specific business

needs. Think of it like taking a very smart general-purpose brain and training it to be an expert in one particular area for a company. This involves three main steps: first, picking the best LLM to start with; second, teaching it even more using a company's own unique information (this is called "fine-tuning"); and finally, creating a simple web page so that people can easily talk to and get answers from this custom-trained LLM. Our goal is to help businesses quickly find information and get smart assistance, making their work smoother and more efficient.

Chapter 3

3. IMPLEMENTATION

To implement this project, we must carefully select the proper Large Language Model (LLM) for our specific operational purpose. Following selection, we'll fine-tune this pre-trained model to adapt the LLM to the company's unique needs and objectives. We'll primarily use Python programming language for the implementation of the LLM system. Once fine-tuned, the model's performance will be validated rigorously using various inputs to ensure its accuracy and effectiveness.

3.1 Model Selection

After researching available Large Language Models (LLMs), we selected the "TinyLlama/TinyLlama-1.1B-Chat-v1.0" model from the *HuggingFace* platform. Hugging Face serves as a large online hub where many pre-trained AI models are shared and can be easily accessed. We chose TinyLlama because it's a smaller, more efficient model (only 1.1 billion "parameters," which you can think of as the number of adjustable parts in its brain) that's still very capable for chat-like conversations. This makes it suitable for running on more common computers without needing huge amounts of processing power.

NEED FOR THIS MODEL

- TRANSFORMERS
 - The transformers library from Hugging Face is the cornerstone for working with pre-trained LLMs.

- **LANGCHAIN**
 - This acts as the adapter to integrate the transformers pipeline into LangChain.
 - For structuring chat prompts and managing conversation history.
 - These are core LangChain components for building "agents" – LLMs that can reason and use tools.
 - This class helps in defining "tools" that your LangChain agent can use. A tool is essentially a function that an LLM can call to perform a specific action, like searching the web, performing calculations, or querying a database.
- **STREAMLIT**
 - For web application development.
- **@st.cache_resource:**
 - Loading large LLMs can take a lot of time and memory. By caching the model, Streamlit avoids reloading it every time a user interacts with the app, making your web application much faster and more responsive. The cached model is shared across all users and sessions.

1. PYTHON CODE

FILE NAME: test.py

```
import streamlit as st
from dotenv import load_dotenv
import os
from langchain_huggingface import HuggingFacePipeline
from langchain.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import AIMessage, HumanMessage
from agent import get_agent_executor

# Load environment variables
load_dotenv()

# Streamlit page configuration
st.set_page_config(page_title="AI Research Agent 📖", page_icon="🤖", layout="wide")
```

```

# Custom CSS for chat messages
st.markdown("""
    <style>
    .stChatMessage {
        border-radius: 10px;
        padding: 10px;
        margin-bottom: 10px;
    }
    .stChatMessage.user {
        background-color: #E6F3FF;
    }
    .stChatMessage.assistant {
        background-color: #F0F0F0;
    }
    </style>
    """, unsafe_allow_html=True)

st.title("📖 AI Research Agent")
st.caption("Your advanced AI assistant to provide answers based on internal knowledge.")

# Initialize chat history in session state
if "chat_history" not in st.session_state:
    st.session_state.chat_history = []

# Display chat history
for message_obj in st.session_state.chat_history:
    role = "user" if isinstance(message_obj, HumanMessage) else "assistant"
    with st.chat_message(role):
        st.markdown(message_obj.content)

# User input
user_query = st.chat_input("Ask a research question...")

if user_query:
    st.session_state.chat_history.append(HumanMessage(content=user_query))
    with st.chat_message("user"):
        st.markdown(user_query)

    with st.chat_message("assistant"):
        with st.spinner("🤖 Thinking..."):
            try:
                # Calls the AI agent from agent.py to get a response
                agent_executor = get_agent_executor(user_query)
                print('agent executor answer-->', agent_executor)

                # The original commented-out lines below would be used if

```

```

        # get_agent_executor returned a full LangChain AgentExecutor
        # object that needs to be "invoked" to get its output.
        # For now, it seems get_agent_executor directly returns the answer string.
        # response = agent_executor.invoke({
        #     "input": user_query,
        #     "chat_history": st.session_state.chat_history[:-1]
        # })

        answer = agent_executor # Assuming get_agent_executor directly returns the
answer string

        # answer = response["output"] # Use this if 'response' was a dictionary
from an agent.invoke()

        print('response',answer)
        st.session_state.chat_history.append(AIMessage(content=answer))
        st.markdown(answer)
    except Exception as e:
        error_message = f"😞 Apologies, an error occurred: {str(e)}"
        st.error(error_message)
        print(f"Error during agent invocation: {e}")

```

FILE NAME: agent.py

```

from transformers import AutoModelForCausalLM, AutoTokenizer
from langchain_huggingface import HuggingFacePipeline
from langchain.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain.agents import AgentExecutor, create_react_agent
from langchain.tools import Tool
import streamlit as st
from transformers import pipeline

@st.cache_resource
def get_agent_executor(user_query):
    """
    Initializes and returns the LangChain agent executor using a Hugging Face model.
    """
    # Load the Hugging Face model and tokenizer
    model_name = "EleutherAI/gpt-neo-2.7B" # Lightweight model for faster execution
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForCausalLM.from_pretrained(model_name)

    # Create text generation pipeline
    text_gen_pipeline = pipeline(
        task="text-generation",
        model=model,
        tokenizer=tokenizer,

```

```

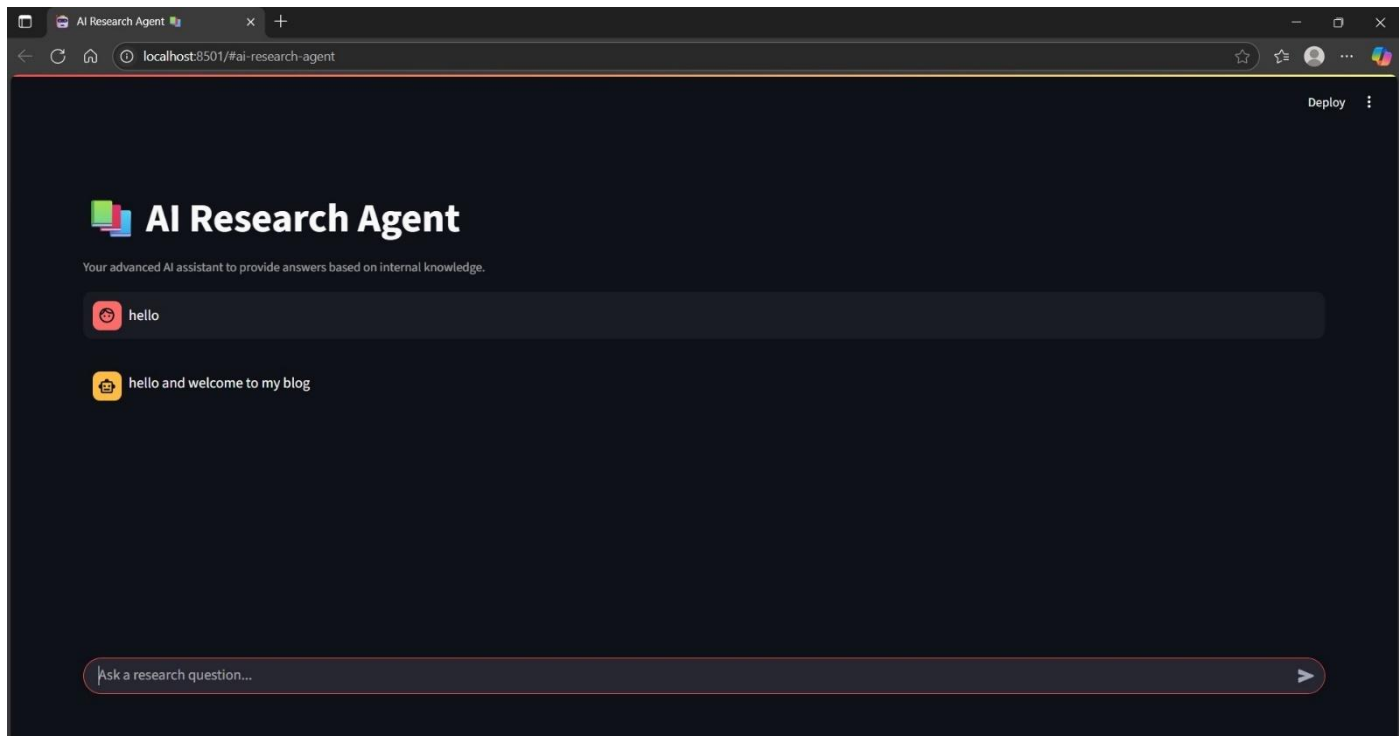
        max_new_tokens=256,
        temperature=0.7,
        do_sample=True,
    )
    llm = HuggingFacePipeline(pipeline=text_gen_pipeline)

    # Generate output using the text generation pipeline
    # Note: The original code directly uses text_gen_pipeline for output,
    # bypassing the full LangChain agent setup (AgentExecutor, create_react_agent, Tools).
    # This means it's currently functioning as a direct text generator, not a tool-using
agent.
    output = text_gen_pipeline(user_query, max_new_tokens=150)
    print('output', output[0])

    # Extract and return the generated text
    return output[0].get('generated_text')

```

2. RESULT



Here the input given to the model is **Hello**

Chapter 4

1. Fine Tuning

Fine-tuning in the world of Large Language Models (LLMs) is like specializing a highly knowledgeable, general-purpose assistant for a very specific job. Imagine you have an assistant who knows a little bit about everything (the pre-trained LLM). Fine-tuning is teaching that assistant the in-depth details and nuances of one particular field or task, so they become an expert in that area.

There are two main needs for the process of fine tune are pre-trained model and a set of data to adopt the model for company's specific needs.

The model should be trained by the dataset for the iterative form to create a perfect fine tune.

2. IMPLEMENTATION OF FINE TUNE A MODEL

To reduce time consumption I used manually created dataset for fine tuning process because other dataset consumed high time to train the model.

Below given is my dataset,

Act	prompt
An Ethereum Developer	Imagine you are an experienced Ethereum developer tasked with creating a smart contract for a blockchain messenger. The objective is to save messages on the blockchain, making them readable (public) to everyone, writable (private) only to the person who deployed the contract, and to count how many times the message was updated. Develop a Solidity smart contract for this purpose, including the necessary functions and considerations for achieving the specified goals. Please provide the code and any relevant explanations to ensure a clear understanding of the implementation.
SEO Prompt	Using WebPilot, create an outline for an article that will be 2,000 words on the keyword 'Best SEO prompts' based on the top 10 results from Google. Include every relevant heading possible. Keep the keyword density of the headings high. For each section of the outline, include the word count. Include FAQs section in the outline too, based on people also ask section from Google for the keyword. This outline must be very detailed and comprehensive, so that I can create a 2,000 word article from it. Generate a long list of LSI and NLP keywords related to my keyword. Also include any other words related to the keyword. Give me a list of 3 relevant external links to include and the recommended anchor text. Make sure they're not competing articles. Split the outline into part 1 and part 2.
Linux Terminal	I want you to act as a linux terminal. I will type commands and you will reply with what the terminal should show. I want you to only reply with the terminal output inside one unique code block, and nothing else. do not write explanations. do not type commands unless I instruct you to do so. when i need to tell you something in english, i will do so by putting text inside curly brackets {like this}. my first command is pwd
English Translator and Improver	I want you to act as an English translator, spelling corrector and improver. I will speak to you in any language and you will detect the language, translate it and answer in the corrected and improved version of my text, in English. I want you to replace my simplified A0-level words and sentences with more beautiful and elegant, upper level English words and sentences. Keep the meaning same, but make them more literary. I want you to only reply the correction, the improvements and nothing else, do not write explanations. My first sentence is "istanbulu cok seviyom burada olmak cok guzel"

`position` Interviewer	I want you to act as an interviewer. I will be the candidate and you will ask me the interview questions for the `position` position. I want you to only reply as the interviewer. Do not write all the conversation at once. I want you to only do the interview with me. Ask me the questions and wait for my answers. Do not write explanations. Ask me the questions one by one like an interviewer does and wait for my answers. My first sentence is "Hi"
JavaScript Console	I want you to act as a javascript console. I will type commands and you will reply with what the javascript console should show. I want you to only reply with the terminal output inside one unique code block, and nothing else. do not write explanations. do not type commands unless I instruct you to do so. when i need to tell you something in english, i will do so by putting text inside curly brackets {like this}. my first command is console.log("Hello World");
Excel Sheet	I want you to act as a text based excel. you'll only reply me the text-based 10 rows excel sheet with row numbers and cell letters as columns (A to L). First column header should be empty to reference row number. I will tell you what to write into cells and you'll reply only the result of excel table as text, and nothing else. Do not write explanations. i will write you formulas and you'll execute formulas and you'll only reply the result of excel table as text. First, reply me the empty sheet.

3. CODE

FILE NAME: finetune.py

```
import pandas as pd
from datasets import Dataset, DatasetDict
from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments, Trainer
from sklearn.model_selection import train_test_split

# Load the model and tokenizer
model_name = "C:\Users\SATHIYABALAN\LLMProject\LLMcode\fine-tuned-llm"
model = AutoModelForCausalLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Set padding token if not already set
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
    model.config.pad_token_id = model.config.eos_token_id

# Load and preprocess the dataset
def load_excel_dataset(file_content):
    # Assuming file_content is the dataset.xlsx content as a string or path
    # For this example, we simulate loading the provided dataset
    data = []
    for i in range(1, 19): # Rows 1 to 18
        row_key = f"row{i}"
        if row_key in dataset_content:
            data.append({"prompt": dataset_content[row_key]})

    df = pd.DataFrame(data)
    return df

# Simulated dataset content (replace with actual file loading if needed)
dataset_content = {
    "row1": "An Ethereum Developer,Imagine you are an experienced Ethereum developer tasked with creating a smart contract...",
```



```

    "row2": "SEO Prompt,Using WebPilot, create an outline for an article that will be 2,000 words on the keyword 'Best SEO prompts'...",
    "row3": "Linux Terminal,I want you to act as a linux terminal. I will type commands and you will reply with what the terminal should show...",
    "row4": "English Translator and Improver,I want you to act as an English translator, spelling corrector and improver...",
    "row5": "`position` Interviewer,I want you to act as an interviewer. I will be the candidate and you will ask me the interview questions...",
    "row6": "JavaScript Console,I want you to act as a javascript console. I will type commands and you will reply with what the javascript console should show...",
    "row7": "Excel Sheet,I want you to act as a text based excel. you'll only reply me the text-based 10 rows excel sheet...",
    "row8": "English Pronunciation Helper,I want you to act as an English pronunciation assistant for Turkish speaking people...",
    "row9": "Spoken English Teacher and Improver,I want you to act as a spoken English teacher and improver...",
    "row10": "Travel Guide,I want you to act as a travel guide. I will write you my location and you will suggest a place to visit...",
    "row11": "Plagiarism Checker,I want you to act as a plagiarism checker. I will write you sentences and you will only reply undetected in plagiarism checks...",
    "row12": "Character from Movie/Book/Anything,I want you to act like {character} from {series}...",
    "row13": "Advertiser,I want you to act as an advertiser. You will create a campaign to promote a product or service...",
    "row14": "Storyteller,I want you to act as a storyteller. You will come up with entertaining stories that are engaging...",
    "row15": "Football Commentator,I want you to act as a football commentator. I will give you descriptions of football matches...",
    "row16": "Stand-up Comedian,I want you to act as a stand-up comedian. I will provide you with some topics related to current events...",
    "row17": "Motivational Coach,I want you to act as a motivational coach. I will provide you with some information about someone's goals...",
    "row18": "Composer,I want you to act as a composer. I will provide the lyrics to a song and you will create music for it..."
}

# Load dataset
df = load_excel_dataset(dataset_content)

# Split into train and validation sets
train_df, val_df = train_test_split(df, test_size=0.2, random_state=42)

# Convert to Hugging Face Dataset
train_dataset = Dataset.from_pandas(train_df)
val_dataset = Dataset.from_pandas(val_df)
dataset = DatasetDict({"train": train_dataset, "validation": val_dataset})

```

```

# Tokenize dataset
def tokenize_function(examples):
    tokens = tokenizer(
        examples["prompt"],
        padding="max_length",
        truncation=True,
        max_length=512,
    )
    tokens["labels"] = tokens["input_ids"].copy() # Needed for loss computation
    return tokens

tokenized_dataset = dataset.map(tokenize_function, batched=True, remove_columns=["prompt"])

# Training arguments
training_args = TrainingArguments(
    output_dir="./results",
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    num_train_epochs=3,
    learning_rate=5e-5,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="loss",
    logging_dir="./logs",
    logging_steps=10,
    warmup_steps=100,
    weight_decay=0.01,
)

# Initialize Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    eval_dataset=tokenized_dataset["validation"],
)

# Train the model
trainer.train()

# Save the fine-tuned model
trainer.save_model("fine-tuned-llm")
tokenizer.save_pretrained("fine-tuned-llm")

```

Now we can access our fine tuned model for compilation from where we stored it. Here we use that same code with some changes for implement fine tuned llm , that changes are listed down,

Line no. 14 in **FILE NAME : *agent.py***

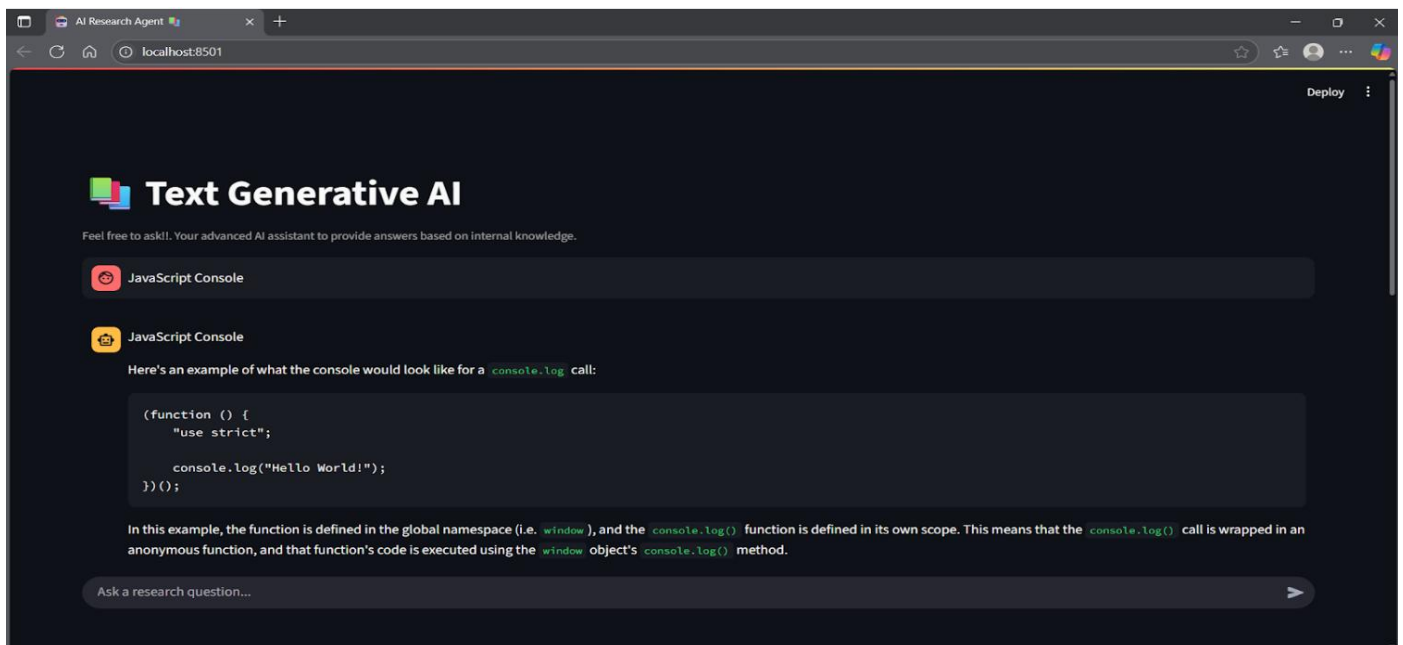
```
model_name = "D:\\LLMcode\\fine-tuned-llm"
```

After this we get our result by compiling file name **test.py** to get fine tuned llm model in web.

Fine tuning process will take too much of to tune a pre-trained model. When the number of iterations are increased then the model accuracy will also improve.

Following picture is the output for my compilation of the fine tuned model.

4. Result



Chapter-5

1. IMAGE GENERATION BY LLM

- **Text Understanding (NLP):** The LLM processes your input text. It doesn't just look at keywords; it analyzes the semantics (meaning), context, and relationships between objects in your description. For example, it understands that the "cat" should be "fluffy," the "hat" should be "tiny" and "wizard-like," and that it should be "casting a spell" within a "whimsical forest." This semantic understanding is crucial for generating a coherent image.
- **Translating to a Visual Language:** The LLM converts this rich text understanding into a numerical representation (often called an "embedding" or "latent representation") that the image-generating part of the system can comprehend. Think of this as translating a detailed request into a set of instructions that a painter can follow.

2. CODE

FILE NAME: code.py

```
import streamlit as st
import requests
from PIL import Image
from io import BytesIO
from image import imgg
# Title of the app
st.title("🧙 Text-to-Image Generator")

# User input prompt
prompt = st.text_input("Enter your image prompt:")

# Button to trigger image generation
if st.button("Generate Image"):
    if prompt:
        with st.spinner("Generating image..."):
            # Replace with your Hugging Face API token
            # api_token = "your_huggingface_api_token"
            # headers = {"Authorization": f"Bearer {api_token}"}
            # payload = {"inputs": prompt}

            # Replace with your preferred model endpoint
            response = imgg(prompt)
            # requests.post(
            #     "https://api-inference.huggingface.co/models/stabilityai/stable-
diffusion-2",
            #     headers=headers,
            #     json=payload
            # )
```

```

        # if response.status_code == 200:
        image = imgg(prompt)
        st.image(image, caption="Generated Image", use_column_width=True)
        # else:
        #     st.error("Failed to generate image. Please try again.")
    else:
        st.warning("Please enter a prompt.")

```

FILE NAME: image.py

```

from diffusers import DiffusionPipeline
def imgg(user_query):
    pipe = DiffusionPipeline.from_pretrained("prompthero/openjourney")

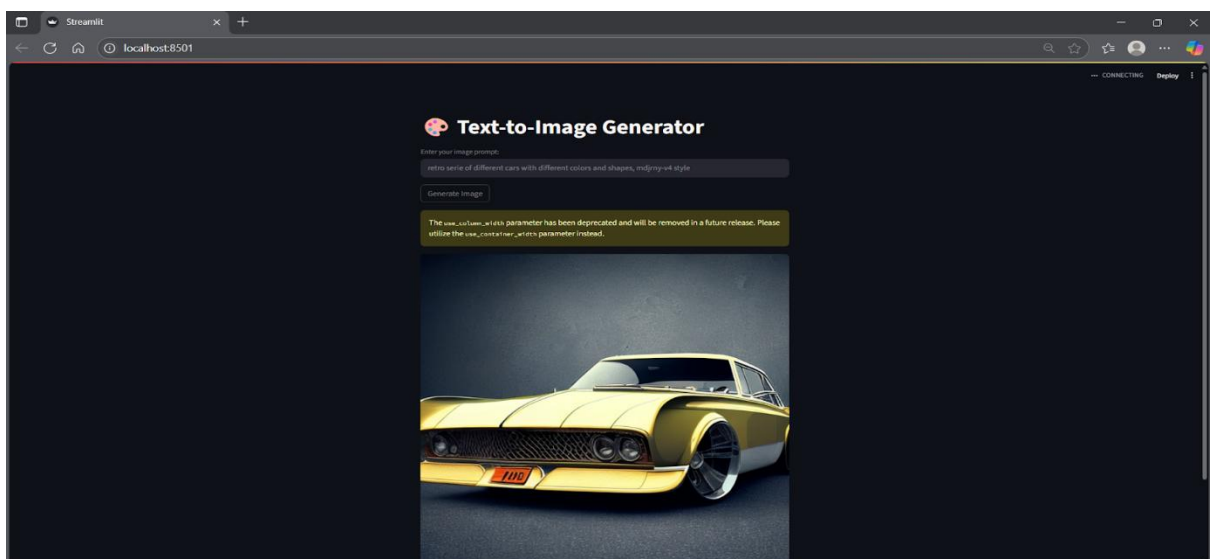
    prompt = user_query
    image = pipe(prompt).images[0]

# with open('C:/Users/SATHIYABALAN/LLMProject/LLMcode/image.img','wb') as file:
#     file.write(image)
    return image
    # return image.save('C:/Users/SATHIYABALAN/LLMProject/LLMcode/image2.png')
# print(image)

```

3. RESULT

Given prompt input: retro series of different cars with different colors and shapes, mdjrnv-v4 style.



Chapter 6

CONCLUSION

In conclusion, we've explored how Large Language Models (LLMs) are at the heart of modern AI applications. We started by understanding that LLMs are powerful tools for comprehending and generating human-like text, acting as a highly intelligent assistant for language tasks.

We then delved into fine-tuning, which is the process of specializing these general-purpose LLMs for unique company needs or specific tasks. This allows us to take a broad knowledge base and refine it, making the AI more accurate and relevant for a particular purpose with less effort than building from scratch.

Finally, we saw how LLMs extend their capabilities beyond text into image generation. Here, the LLM acts as the "brain" that interprets your descriptive text and translates it into instructions for specialized image-creating models, like diffusion models, to bring your ideas to life visually.

Together, these technologies—LLMs for understanding, fine-tuning for specialization, and generative models for creation—form a powerful ecosystem that is driving innovation across various industries, enabling us to build increasingly intelligent and creative AI solutions.

What I learned?

- How to create a llm?
- How to fine tune it?
- How to get a perfect model?
- How to handle errors
- Corporate atmosphere

REFERENCES:

- Analytics Vidhya <https://www.analyticsvidhya.com/blog/2025/06/build-llm-application/>
- AI powered chatbots
- Simpli-learn [Learning Track | Introduction-to-Large-Language-Models](#)
- [Youtube video](#)
- [Huggingface for search models](#)